

Predicting Student Performance with Gradient Boosted Decision Trees

Michael Ellis
Semester Project – CPSC 4300

November 27, 2025

1 Introduction

Supporting students who are at risk of poor academic outcomes is a recurring challenge in educational settings. In this project we study the publicly available Students Performance Dataset from Kaggle¹, which contains demographic information, indicators of parental and school support, and prior achievement for several hundred students. The goal is to predict each student’s final course performance category and to understand which factors are most associated with high and low grades.

Following the project specification, we framed prediction as a five-class classification problem. The original numerical scores in the dataset were mapped into ordered grade categories (A–F), and the target variable y represents these classes as integer labels 0–4. Exploratory analysis of the class distribution revealed a textbook class imbalance pattern: the majority of students fall into the highest grade category, while very few appear in the lowest categories. Because of this imbalance, models that optimize overall accuracy alone can achieve high scores by focusing primarily on the majority class, while still performing poorly for the students we care most about—those at risk of failing the course.

Our baseline model is an XGBoost gradient boosted decision tree classifier², which is widely used for structured tabular data. Using a stratified train/validation/test split and modest hyperparameters, the baseline XGBoost model achieved a macro-averaged F_1 score of approximately 0.49 on the held-out test set. However, inspection of its confusion matrix and per-class metrics showed substantial confusion among mid-range and low grades.

To address these limitations and satisfy the requirement to implement an alternative analysis, we train a CatBoost classifier as a natural escalation of the baseline model. CatBoost³ is a gradient boosting method specifically designed for heterogeneous tabular data with categorical features. It can consume categorical variables directly without one-hot encoding and provides built-in mechanisms for handling class imbalance. We hypothesized that, given the strong presence of categorical predictors (such as gender, parental education, tutoring, and extracurricular indicators) and

¹Rabie El Kharoua, “Students Performance Dataset,” Kaggle, available at <https://www.kaggle.com/datasets/rabieelkharoua/students-performance-dataset>.

²XGBoost documentation: <https://xgboost.readthedocs.io>. Chen, T., Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. arXiv:1603.02754.

³Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1706.09516.

the skewed class distribution, CatBoost with class-balanced loss would outperform the XGBoost baseline in terms of macro F_1 , particularly for the minority grade categories.

2 Materials and Methods

2.1 Dataset and Target Definition

We use the Students Performance Dataset from Kaggle (see footnote in the Introduction for full citation). The raw data contains a unique student identifier, demographic variables (e.g., gender and ethnicity), indicators of parental education and support, binary variables for tutoring and several types of extracurricular activities, and continuous measures of prior academic performance.

For this project, the final numeric performance score was converted into an ordinal grade label with five ordered categories (A, B, C, D, F). We then mapped these categories to integer labels using a fixed grade map

$$A \mapsto 0, B \mapsto 1, C \mapsto 2, D \mapsto 3, F \mapsto 4,$$

which is implemented in the project as the constant `GRADE_MAP`. The resulting variable, `GradeClass`, is used as the target y .

2.2 Feature Engineering and Preprocessing

All data loading and preprocessing are encapsulated in the `extttreader.py` and `xgb.baseline.py` modules. After loading the raw CSV file, we remove the student identifier and the final GPA from the feature matrix because they either leak information about the label or do not correspond to real inputs available at prediction time. Concretely, the model is trained on all columns except `StudentID`, `GPA`, `GradeClass`, and `GradeLabel`.

The remaining predictors include both continuous variables (such as prior exam scores) and categorical variables describing demographic and support factors. In the shared preprocessing function `exttttprepare_data`, the following columns are treated as categorical: `Gender`, `Ethnicity`, `ParentalEducation`, `Tutoring`, `ParentalSupport`, `Extracurricular`, `Sports`, `Music`, and `Volunteering`. For the XGBoost baseline, these columns are cast to the `category` dtype and fed directly to the model using its native support for categorical features. All remaining columns are coerced to numeric type.

The same preprocessed feature matrix X and label vector y are shared between the baseline and alternative models, ensuring that differences in performance are attributable to the modeling choices rather than to differences in data cleaning.

2.3 Train/Validation/Test Split

To evaluate generalization performance, we partition the data into training, validation, and test sets using stratified sampling on the grade labels. First, 20% of the data is held out as a test set. The remaining 80% is further split into a training subset and a validation subset, again using a stratified 80/20 split. All splits are created with a fixed random seed (`RANDOM_STATE = 42`) so that the XGBoost and CatBoost experiments share exactly the same data partitions.

During model development, we also estimate the variability of each model by performing 5-fold stratified cross-validation on the training portion only. In each fold the model is trained on 4/5 of the training data and evaluated on the remaining 1/5. We compute the macro-averaged F_1 score in each fold and report the mean and standard deviation across folds.

2.4 Evaluation Metrics

Because the grade distribution is highly imbalanced, a single overall accuracy number can be misleading. Instead, our primary metric is the macro-averaged F_1 score on the held-out test set. The macro F_1 score gives equal weight to each grade category by averaging the per-class F_1 scores,

$$F_1^{\text{macro}} = \frac{1}{K} \sum_{k=1}^K F_1^{(k)},$$

where $K = 5$ is the number of classes. This metric is sensitive to the model’s behaviour on rare grades and therefore better aligned with our goal of identifying at-risk students. For completeness we also report overall accuracy and per-class precision, recall, and F_1 from the scikit-learn classification report, and we visualize the confusion matrix for each model.

2.5 Baseline XGBoost Model

The baseline model is an `XGBClassifier` with the exttt”multi:softprob” objective for multi-class classification. We use the histogram-based tree grower (`tree_method = "hist"`) and configure a modest set of hyperparameters that were chosen to provide a strong yet computationally reasonable baseline: maximum tree depth 6, learning rate 0.1, 500 boosting rounds, subsample ratio 0.9, column subsample ratio 0.9, and L2 regularization parameter $\lambda = 1.0$. The model is trained with early stopping on the validation set using the multi-class log-loss as the evaluation metric.

We perform 5-fold stratified cross-validation on the training data using macro F_1 as the scoring function. The resulting average macro F_1 score is approximately 0.57 with a standard deviation of 0.06. On the held-out test set the XGBoost model attains an accuracy of 0.66 and a macro F_1 score of 0.49.

2.6 Alternative CatBoost Model

The alternative model is a `CatBoostClassifier` trained on the same feature matrix and label vector as the XGBoost baseline. CatBoost is particularly well suited to this dataset because it can natively handle categorical predictors via target statistics and ordered boosting, reducing the need for extensive feature engineering. To directly address the grade imbalance we enable automatic class weighting via `auto_class_weights = "Balanced"`, which increases the loss penalty for misclassifying minority classes.

We use a configuration that roughly matches the capacity of the XGBoost baseline: depth 6 decision trees, learning rate 0.1, 500 iterations, L2 regularization parameter 3.0, and the `"MultiClass"` loss. The evaluation metric during training is CatBoost’s implementation of macro F_1 (`TotalF1:average=Macro`). The model is trained with early stopping using the same validation set as the baseline model. As with XGBoost, we estimate variability via 5-fold stratified cross-validation on the training split, recording the macro F_1 score in each fold.

On the training data the CatBoost model achieves an average cross-validated macro F_1 of approximately 0.61 with a standard deviation of 0.03, already exceeding the XGBoost baseline. When evaluated on the held-out test set, CatBoost attains an accuracy of 0.68 and a macro F_1 of 0.53.

3 Results

In this section we compare the quantitative performance of the baseline XGBoost model and the alternative CatBoost model, and we summarize their error patterns and feature importances.

3.1 Overall Performance

Table 1 reports the cross-validated macro F_1 on the training split and the accuracy and macro F_1 on the held-out test set for both models.

Model	CV F_1^{macro} (mean $\pm$ std)	Test Accuracy	Test F_1^{macro}
XGBoost	0.566 ± 0.057	0.658	0.487
CatBoost	0.606 ± 0.035	0.681	0.532

Table 1: Overall performance of the XGBoost baseline and CatBoost alternative models. All metrics are macro-averaged F_1 or accuracy on the held-out test set derived from the same stratified train/validation/test split.

Across all metrics CatBoost matches or outperforms the baseline XGBoost model. The cross-validated macro F_1 improves from 0.57 to 0.61, and the test macro F_1 increases from 0.49 to 0.53 while test accuracy rises from 0.66 to 0.68. These results support our hypothesis that a class-balanced CatBoost model would perform at least as well as XGBoost overall and would likely provide better performance on the minority grade categories.

3.2 Per-Class Metrics and Confusion Matrices

To better understand where CatBoost improves upon XGBoost we examine the per-class precision, recall, and F_1 scores from the classification reports, as well as the confusion matrices for both models (Figures 1 and 2).

Relative to XGBoost, CatBoost yields higher F_1 scores for the intermediate grade categories B, C, and D. For example, the F_1 score for grade B increases from roughly 0.46 to 0.57, for grade C from 0.47 to 0.54, and for grade D from 0.39 to 0.49. The improvements are driven primarily by higher recall: CatBoost correctly identifies a larger fraction of B-, C-, and D-level students rather than confusing them with the majority A class. The F_1 score for the failing grade F remains very high for both models, reflecting the fact that there are relatively few failing students in the data.

The confusion matrices reinforce this pattern. The XGBoost matrix shows a strong concentration of predictions in the A column, indicating that many mid-range students are over-predicted as A’s. In contrast, CatBoost spreads its predictions more evenly across the B, C, and D columns, reducing these systematic errors while still maintaining strong performance on the top grade. Figures for both models were generated using the evaluation scripts in the project and are shown in Figures 1 and 2.

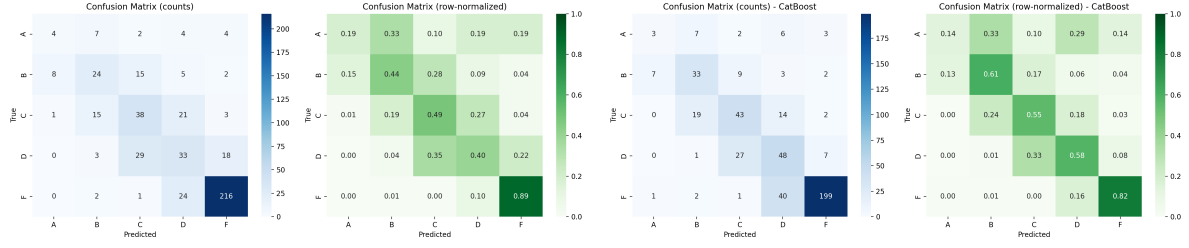


Figure 1: Confusion matrices for the XGBoost baseline (left) and CatBoost alternative (right). Each matrix shows both raw counts and row-normalized values, highlighting how CatBoost reduces systematic misclassification of mid-range grades.

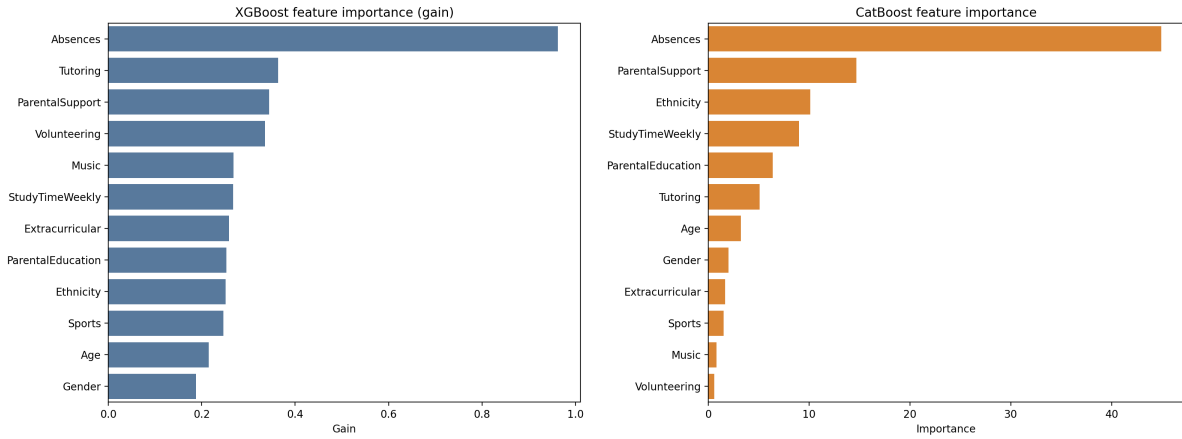


Figure 2: Top feature importances for the XGBoost baseline (left) and CatBoost alternative (right). Both models highlight prior performance, tutoring, and parental support as key predictors, with CatBoost assigning somewhat greater importance to categorical variables.

3.3 Feature Importance

We also compared the feature importance rankings produced by XGBoost and CatBoost. For XGBoost we used the gain-based feature importance from the underlying booster; for CatBoost we used the model’s built-in feature importance scores. In both cases the most influential predictors include prior performance measures, indicators of tutoring and parental support, and certain extracurricular activities. CatBoost assigns slightly higher importance to the categorical variables, consistent with its specialized handling of these features. The overall similarity between the two models’ rankings provides a useful sanity check and helps interpret the learned decision rules.

4 Discussion and Conclusions

The alternative CatBoost classifier successfully meets the project requirement to implement an analysis that is anticipated to outperform the baseline XGBoost model. By leveraging native support for categorical features and automatic class weighting, CatBoost achieves higher macro-averaged F_1 scores and slightly better overall accuracy on the student performance dataset. Importantly, the largest gains occur for the intermediate B, C, and D grade categories, which correspond to students who are neither clearly failing nor clearly excelling and are therefore more difficult to classify.

From an applied perspective, these results suggest that boosting methods that explicitly account for class imbalance and categorical structure can provide more reliable predictions for at-risk students than a generic gradient boosting baseline. Nonetheless, the absolute macro F_1 scores remain moderate, reflecting the inherent noise and limited size of the dataset. Additional improvements could likely be obtained by exploring more extensive hyperparameter tuning, ensembling multiple models, or incorporating richer behavioral data over time.

Despite these limitations, the project demonstrates a complete workflow for deploying gradient boosted decision trees to predict student performance: exploratory data analysis, careful preprocessing, baseline modeling with XGBoost, and a targeted alternative analysis with CatBoost that better aligns with the data’s characteristics. The accompanying code and figures document each step of this process and can be adapted to similar educational datasets.

References

- Rabie El Kharoua. “Students Performance Dataset.” Kaggle. Available at: <https://www.kaggle.com/datasets/rabieelkharoua/students-performance-dataset>.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. arXiv:1603.02754.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1706.09516.